

QQMusic 业务代码深度复习

目标：把当前仓库中的业务源码转化为可以复述、可以面试说明、可以继续扩展的知识体系。

阅读约定：本文按“每个声明、有效语句和状态变化”解释代码。纯空行、花括号、重复的同型 getter/setter 会合并说明，但不会跳过其语义。以仓库当前代码为准；PDF 和 XMind 只作为功能背景。

0. 项目全景：先建立正确心智模型

这是一个基于 Qt 5 Widgets 的本地音乐播放器。QQMusic 既是主窗口，也是当前 Demo 的应用协调器；它持有歌曲集合、数据库连接、播放器、播放列表、音量浮层和歌词页。业务核心不是“点击一个按钮播放一首歌”，而是下面这条状态链：

本地文件 / SQLite

- > MusicList（内存中的全量歌曲）
- > CommonPage（按页面条件筛选歌曲 ID）
- > QMediaPlaylist（当前播放页临时生成的队列）
- > QMediaPlayer（实际播放与状态信号）
- > 进度、封面、歌词、最近播放页面

0.1 模块职责表

模块	主要职责	不应该承担的职责
main.cpp	Qt 应用初始化、单实例、事件循环	歌曲和界面业务
QQMusic	初始化、跨模块协调、信号槽汇总	保存多份歌曲数据
Music	一首歌曲的元数据与持久化记录	控制 UI
MusicList	全部歌曲、路径去重、读写 DB	直接绘制列表
CommonPage	喜欢/本地/最近三种数据投影	真正拥有歌曲
ListItemBox	单行歌曲显示与爱心交互	更新全局数据
LrcPage	解析和显示同步歌词	决定播放时间
MusicSlider	把鼠标位置变成播放比例	调用播放器
VolumeTool	把鼠标位置变成音量/静音意图	直接保存音量状态
btForm / RecBox	侧栏和推荐页视觉交互	管理播放业务

0.2 生命周期与所有权

Qt 的 QObject 父子树会自动释放子对象。QQMusic 以 this 为 parent 创建 QMediaPlayer、QMediaPlaylist、VolumeTool、LrcPage、托盘对象和动画，因此主窗口析构时会递归释放它们。ui 则由 QQMusic::~~QQMusic() 手动 delete。

这个项目的数据所有权只有一处：`QQMusic::musicList`。页面只保留歌曲 ID，播放列表只保留当前播放来源页的 URL 队列。理解这一点，才能理解收藏、历史记录为什么能跨页面同步。

1. `main.cpp`：启动边界

源码：[main.cpp](#)

```
#include "qqmusic.h"
#include <QMessageBox>
#include <QSharedMemory>
#include <QApplication>
```

1. `qqmusic.h` 引入主窗口类型，否则后面的 `QQMusic w;` 无法编译。
2. `QMessageBox` 用于第二次启动时提示。
3. `QSharedMemory` 用于进程间共享内存；本项目把它当“单实例锁”，不存业务数据。
4. `QApplication` 是 Widgets 应用的根对象，负责事件分发、窗口管理、平台集成和事件循环。

```
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
```

1. `main` 是原生 C++ 入口，`argc/argv` 是命令行参数。
2. `QApplication a(argc, argv)` 必须先于任何 `QWidget` 构造；Qt 会读取并可能消费 Qt 相关的命令行参数。
3. `a` 位于栈上，但其生命周期覆盖整个事件循环，因此所有窗口都能依赖它。

```
QSharedMemory shareMemory("QQMusic");
```

1. 构造一个共享内存句柄，键名是全局约定的 `QQMusic`。
2. 不同进程使用同一个键时，会看到同一块系统共享内存。
3. 键名是单实例机制的本体；改名会让新旧两个版本能够同时运行。

```
if(shareMemory.attach())
{
    QMessageBox::information(nullptr, "QQMusic提示", "QQMusic已经在运行");
    return 0;
}
```

1. `attach()` 尝试连接**已经存在**的共享内存。
2. 返回 `true` 表示前一个实例已创建该内存，因此本进程是第二实例。
3. `nullptr` 表示消息框没有父窗口，因为主窗口尚未构造。
4. `return 0` 立即结束第二实例，避免同时读写同一数据库和媒体状态。
5. 注意：`attach()` 失败不只可能表示“不存在”，也可能是权限或系统错误；Demo 将所有失败都当成“可启动”。

```

shareMemory.create(1);
QQMusic w;
w.show();
return a.exec();
}

```

1. `create(1)` 分配 1 字节即可，字节内容不重要，只要能作为存在标记。
2. 当前代码没有判断 `create` 返回值；生产代码应在失败时处理错误。
3. `QQMusic w` 在栈上构造主窗口，离开 `main` 时自动析构。
4. `w.show()` 请求显示窗口；真实绘制在事件循环运行后发生。
5. `a.exec()` 启动事件循环。用户点击、动画、播放器回调、重绘和定时事件都由它调度。
6. `exec()` 返回意味着应用退出，返回值交给操作系统。

架构沉淀

入口文件只负责“运行环境”和“进程规则”，不放业务逻辑。这是健康的 Qt 项目结构。单实例实现简洁，但应补充 `create()` 的失败判断，并在异常情况下输出 `shareMemory.errorString()`。

2. QQMusic 头文件：总控对象持有什么状态

源码： [qgmusic.h](#)

```

class QQMusic : public QWidget
{
    Q_OBJECT

```

1. 继承 `QWidget`，所以 `QQMusic` 本身就是窗口。
2. `Q_OBJECT` 启用 Qt 元对象系统；没有它，`signals`、`slots`、运行时类型信息和字符串式连接都不可用。

```

public:
    QQMusic(QWidget *parent = nullptr);
    ~QQMusic();

```

1. `parent = nullptr` 允许它作为顶层窗口创建。
2. 析构函数负责删除非 `QObject` 的 `ui` 管理对象。

```

void initUI();
void initSqlite();
void initMusicList();
void initPlayer();
void connectSignalAndSlots();

```

这五个函数按依赖顺序拆分构造函数：先有控件，再有数据库，再恢复数据，再创建播放器，最后连接事件。它们虽然是 `public`，但从封装角度更适合设为 `private`，因为外部对象不应重新初始化主窗口。

```

QPoint dragPosition;
SqliteDatabase sqlite;
MusicList musicList;
VolumeTool *volumeTool;
LrcPage* lrcPage;
QPropertyAnimation* lrcAnimal;
QMediaPlayer* player;
QMediaPlaylist* playList;
CommonPage* currentPage;
qint64 totalTime;
int currentIndex;
bool isDrag;

```

1. `dragPosition` 保存鼠标相对窗口左上角的位置，用于无边框拖拽。
2. `sqlite` 是默认 SQLite 连接的句柄。
3. `musicList` 是唯一的全量歌曲集合，按值作为成员拥有。
4. `volumeTool`、`lrcPage`、`lrcAnimal`、`player`、`playList` 都在初始化中 `new`，并由 `this` 管理生命周期。
5. `currentPage` 代表**当前播放来源页**，不是当前可见页。
6. `totalTime` 单位为毫秒，给时间显示和 `seek` 使用。
7. `currentIndex` 是当前媒体在播放列表内的索引；`-1` 表示无有效当前项。
8. `isDrag` 表示窗口是否正由鼠标拖动。
9. 代码只初始化了 `currentIndex`，`totalTime` 和 `isDrag` 未初始化，这是必须修正的潜在未定义行为。

3. QQMusic 构造、析构与初始化

源码：[qqmusic.cpp](#)

```

QQMusic::QQMusic(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::QQMusic)
    , currentIndex(-1)
{
    ui->setupUi(this);
    initUI();
    initSqlite();
    initMusicList();
    initPlayer();
    connectSignalAndSlots();
}

```

1. `QWidget(parent)` 先构造父类窗口部分。
2. `ui(new Ui::QQMusic)` 创建 Designer 生成的界面管理器；这时子控件尚未创建。
3. `currentIndex(-1)` 明确表示启动时没有歌曲在播放。
4. `setupUi(this)` 根据 `.ui` 文件创建所有控件、布局和提升后的自定义控件，并将它们挂入 `this` 的对象树。
5. `initUI()` 只能在 `setupUi` 后调用，因为它访问 `ui->play`、`ui->localPage` 等控件。
6. `initSqlite()` 先建连接和表，保证随后读取合法。
7. `initMusicList()` 从 DB 恢复歌曲并填充页面。

8. `initPlayer()` 创建播放器和列表。
9. `connectSignalAndSlots()` 最后连接，因为所有信号源、接收者均已存在。

```
QQMusic::~~QQMusic()
{
    delete ui;
}
```

1. `ui` 不是 `QObject`，因此需要手动释放。
2. `ui` 销毁关联的界面控件。
3. 其余以 `this` 为 parent 的 `QObject` 子对象由 `QWidget/QObject` 析构链自动处理，不应重复 `delete`。

架构沉淀

构造顺序反映依赖方向，值得在面试中说明。实际改进是将 `totalTime(0)`、`isDrag(false)` 加入初始化列表，并让 `initSqlite()` 返回 `bool`，失败时阻止后续数据库相关初始化。

4. `QQMusic::initUI()`：无边框窗口、侧栏、浮层和推荐页

```
this->setWindowFlag(Qt::FramelessWindowHint);
setAttribute(Qt::WA_TranslucentBackground);
setWindowIcon(QIcon(":/images/tubiao.png"));
ui->max->setEnabled(false);
```

1. `FramelessWindowHint` 去掉原生标题栏，所以系统不会提供拖动、关闭等默认交互。
2. `WA_TranslucentBackground` 允许透明像素；窗口阴影和圆角需要这个前提。
3. `setWindowIcon` 设置任务栏/窗口图标，`:/` 是 Qt Resource 路径。
4. `max` 按钮被禁用，实现“不可最大化”的需求。

```
QSystemTrayIcon* trayIcon = new QSystemTrayIcon(this);
trayIcon->setIcon(QIcon(":/images/tubiao.png"));
trayIcon->show();
QMenu* trayMenu = new QMenu(this);
trayMenu->addAction("显示", this, &QWidget::showNormal);
trayMenu->addSeparator();
trayMenu->addAction("退出", this, &QQMusic::onQQMusicQuit);
trayIcon->setContextMenu(trayMenu);
```

1. 托盘和菜单以 `this` 为 parent，主窗口销毁时自动销毁。
2. `show()` 将图标注册到系统托盘。
3. “显示”直接调用继承来的 `showNormal()`，可从隐藏/最小化状态恢复窗口。
4. “退出”刻意不调用 `close()`，而是调用项目槽 `onQQMusicQuit()`，以便先写数据库。
5. `setContextMenu()` 把菜单交给托盘右键处理。
6. 资源路径写成 `:/images/tubiao.png`，建议改成 `:/images/tubiao.png`，与其他资源路径一致。

```
QGraphicsDropShadowEffect* shadowEffect = new QGraphicsDropShadowEffect(this);
shadowEffect->setOffset(0, 0);
shadowEffect->setColor("#000000");
shadowEffect->setBlurRadius(10);
this->setGraphicsEffect(shadowEffect);
```

1. 创建窗口级阴影效果。
2. `offset(0, 0)` 表示四周均匀扩散，而不是右下偏移。
3. 字符串颜色会转换为 `QColor`。
4. 模糊半径为 10；半径过大时透明边缘会影响歌词窗口几何动画的视觉效果。
5. `setGraphicsEffect` 后该效果由窗口接管。

```
ui->rec->setIconAndText("...rec.png", "推荐", 0);
ui->radio->setIconAndText("...radio.png", "电台", 1);
ui->music->setIconAndText("...music.png", "音乐馆", 2);
ui->like->setIconAndText("...like.png", "我喜欢", 3);
ui->local->setIconAndText("...local.png", "本地下载", 4);
ui->recent->setIconAndText("...recent.png", "最近播放", 5);
```

1. 六行都调用 `btForm` 的统一接口，写入图标、文字和目标页 ID。
2. ID 必须与 `.ui` 中 `QStackedWidget` 页序一致：推荐 0、电台 1、音乐馆 2、喜欢 3、本地 4、最近 5。
3. 这是一种“数据驱动导航”的简化写法：同一套按钮逻辑只因 ID 不同而跳转不同页。
4. 缺点是魔法数字与 UI 页序强耦合，重排 Designer 页面后不会有编译错误，却会跳转错页。

```
ui->local->showAnimat();
ui->stackedWidget->setCurrentIndex(4);
currentPage = ui->localPage;
```

1. 默认显示本地页对应的侧栏动态条。
2. `setCurrentIndex(4)` 让本地页成为初始可见页。
3. `currentPage` 初始也设为本地页；后续它代表“播放来源”，不是浏览页面。

```
srand((unsigned int)time(nullptr));
ui->recMusicBox->initRecBoxUi(randomPicture(), 1);
ui->supplyMusicBox->initRecBoxUi(randomPicture(), 2);
```

1. `srand` 为 C 随机数生成器播种。
2. `randomPicture()` 内部使用 `std::random_shuffle`，Qt 5/C++11 下它通常仍依赖 `rand()`，故种子有效。
3. 参数 1 表示推荐区的一行布局；参数 2 表示补给站的两行布局。

```
ui->likePage->setCommonPageUI("我喜欢", ":/images/ilikebg.png");
ui->localPage->setCommonPageUI("本地下载", ":/images/localbg.png");
ui->recentPage->setCommonPageUI("最近播放", ":/images/recentbg.png");
```

三个页面同属 `CommonPage`，只注入不同标题和背景。这是该 Demo 最清晰的复用点之一。

```

ui->play->setIcon(QIcon(":/images/play3.png"));
ui->playMode->setIcon(QIcon(":/images/shuffle_2.png"));
volumeTool = new VolumeTool(this);
lrcPage = new LrcPage(this);
lrcPage->setGeometry(10, 10, lrcPage->width(), lrcPage->height());
lrcPage->hide();

```

1. 前两行给底部按钮设置初始图标；实际播放状态变化后会由播放器信号覆盖。
2. `VolumeTool` 与 `LrcPage` 都以主窗口为 owner。
3. 歌词页初始放在窗口 (10, 10)，保留其 Designer 宽高。
4. `hide()` 保证启动时歌词页不显示。

```

lrcAnimal = new QPropertyAnimation(lrcPage, "geometry", this);
lrcAnimal->setDuration(500);
lrcAnimal->setStartValue(QRect(10, 10 + lrcPage->height(), ...));
lrcAnimal->setEndValue(QRect(10, 10, ...));

```

1. 动画目标是 `lrcPage` 的可写 Qt 属性 `geometry`。
2. 起点 Y 坐标在歌词页自身高度之下，视觉上位于窗口下方。
3. 终点回到 (10, 10)，形成向上滑入。
4. 动画只配置不启动，启动动作在 `onLrcwordClicked()`。

架构沉淀

`initUI()` 专注“创建与配置”，没有直接混入歌曲筛选或播放逻辑。改善方向是将页面 ID 抽成枚举、修正托盘资源路径，并避免让全部侧栏动画在不可见时仍无限运行。

5. SQLite 初始化、数据恢复与通用刷新

```

sqlite = QSqlDatabase::addDatabase("QSQLITE");
sqlite.setDatabaseName("QQMusic.db");
if(!sqlite.open()) { ... return; }

```

1. `addDatabase("QSQLITE")` 建立默认 SQLite 连接。
2. 数据库名是相对路径，实际 DB 所在目录取决于程序启动目录，通常是可执行文件目录。
3. `open()` 失败会弹窗并返回；但构造函数仍继续执行后续初始化，这是当前的错误传播缺口。

```

QString sql = "CREATE TABLE IF NOT EXISTS MusicInfo(...)";
QSqlQuery query;
if(!query.exec(sql)) { ... return; }

```

1. `IF NOT EXISTS` 使首次运行建表、后续运行保持表不变。
2. `musicId` 加 `UNIQUE`，保证 UUID 不重复。
3. 静态信息有歌名、歌手、专辑、URL、时长；可变用户状态有喜欢和历史。
4. `QSqlQuery` 未指定连接名，使用默认连接。


```
musicList.readFromDB();
ui->likePage->setPageType(LIKE_PAGE);
ui->likePage->refresh(musicList);
ui->localPage->setPageType(LOCAL_PAGE);
ui->localPage->refresh(musicList);
ui->recentPage->setPageType(HISTORY_PAGE);
ui->recentPage->refresh(musicList);
```

1. 首先将 DB 行还原为 `Music` 对象。
2. 然后为三个通用页注入不同筛选类型。
3. 每个 `refresh()` 都从同一个 `musicList` 重新投影 UI；页面不拥有独立歌曲副本。

架构沉淀

“全量集合 + 多个筛选视图”比“每页各存一个歌曲容器”更容易保证一致性。代价是每次状态改变会重建列表，数据量很大时应改用 Model/View。

6. 播放器初始化与信号槽注册

```
player = new QMediaPlayer(this);
playlist = new QMediaPlaylist(this);
playlist->setPlaybackMode(QMediaPlaylist::Random);
player->setPlaylist(playlist);
player->setVolume(20);
```

1. `player` 负责解码、播放、暂停、seek 和媒体元数据。
2. `playlist` 负责媒体队列、当前索引和播放模式。
3. 实际默认模式是 `Random`，与“默认列表循环”的注释不一致，复习时以代码为准。
4. `setPlaylist` 把队列交给播放器；之后 `next()`、`previous()` 改变的就是该队列。
5. Qt 5 音量范围是 0 到 100，初始值 20。

```
connect(player, &QMediaPlayer::stateChanged, this,
        &QQMusic::onPlayStateChanged);
connect(player, &QMediaPlayer::durationChanged, this,
        &QQMusic::onDurationChanged);
connect(player, &QMediaPlayer::positionChanged, this,
        &QQMusic::onPositionChanged);
connect(player, &QMediaObject::metaDataAvailableChanged, this,
        &QQMusic::onMediaAvailableChanged);
connect(playlist, &QMediaPlaylist::playbackModeChanged, this,
        &QQMusic::onPlaybackModeChanged);
connect(playlist, &QMediaPlaylist::currentIndexChanged, this,
        &QQMusic::onPlayCurrentIndexChanged);
```

1. 这是编译器检查的新式连接语法；信号和槽签名不兼容时会编译失败。
2. `stateChanged` 仅负责播放按钮图标。
3. `durationChanged` 提供总时长，供时间显示和进度计算。
4. `positionChanged` 是持续更新源，驱动当前时间、进度条和歌词。
5. `metaDataAvailableChanged` 在媒体元数据准备好后更新名称、歌手、封面、LRC。
6. `playbackModeChanged` 更新模式图标。

7. `currentIndexChanged` 将新曲标记进历史记录。

`connectSignalAndSlots()` 后续连接的是 UI 意图：六个 `btForm` 连接到 `onBtFormClicked`，三个 `CommonPage` 的收藏信号连接到 `updateLikeMusicAndPage`，播放控制按钮连接到对应播放槽，三个页面的“全部播放/双击歌曲”连接到播放入口，音量、歌词按钮和进度条各自连接到协调槽。其共同原则是：控件发意图，主窗口协调状态，播放器发真实状态。

7. QQMusic：导航、收藏、窗口和本地导入

```
void QQMusic::updateBtFormAnimal()
{
    int index = ui->stackedWidget->indexOf(currentPage);
    if(index == -1) { qDebug() << "该页面不存在"; return; }
    QList<btForm*> btForms = this->findChildren<btForm*>();
    for(auto& btItem : btForms)
        if(index == btItem->getPageId()) btItem->showAnimat();
        else btItem->hideAnimat();
}
```

1. `indexOf(currentPage)` 反查播放来源页的 stack 索引。
2. `-1` 表示该对象不属于这个 `QStackedWidget`，继续执行没有意义。
3. `findChildren<btForm*>()` 利用 `QObject` 树递归找到主窗口下所有侧栏按钮。
4. 遍历中，页 ID 匹配的按钮显示动态条，其余隐藏。
5. 这里同步的是“正在播放来源”的动画，不一定是用户当前浏览页；这是刻意设计，但变量名需要读者特别注意。

```
QJsonArray QQMusic::randomPicture()
{
    QVector<QString> vecImageName;
    vecImageName << "001.png" << ... << "040.png";
    std::random_shuffle(vecImageName.begin(), vecImageName.end());
    QJsonArray objArray;
    for(int i = 0; i < vecImageName.size(); i++) {
        QJsonObject obj;
        obj.insert("path", ":/images/rec/" + vecImageName[i]);
        QString strText = QString("推荐-%1").arg(i, 3, 10, QChar('0'));
        obj.insert("text", strText);
        objArray.append(obj);
    }
    return objArray;
}
```

1. `QVector<QString>` 保存资源文件名，代码中列出 001、003 到 040，实际跳过了 002。
2. `random_shuffle` 就地改变向量顺序；Qt 工程配置为 C++11 时可用，但它在 C++17 已移除，应改为 `std::shuffle` 配合 `std::mt19937`。
3. `QJsonArray/QJsonObject` 在这里不是网络 JSON，而是方便给 `RecBox` 传递 `{path, text}` 的动态结构。
4. `arg(i, 3, 10, QChar('0'))` 将整数格式化为三位十进制，例如 `0 -> 000`。
5. 标签编号跟随机后的数组索引走，而不是原图片文件名；所以“推荐-000”并不固定对应 `001.png`。

```

void QQMusic::updateLikeMusicAndPage(bool isLike, const QString &musicId)
{
    auto it = musicList.findMusicById(musicId);
    if(it != musicList.end()) it->setIsLike(isLike);
    ui->likePage->reFresh(musicList);
    ui->localPage->reFresh(musicList);
    ui->recentPage->reFresh(musicList);
}

```

1. `findMusicById` 返回 `QVector<Music>::iterator`，所以 `it` 直接指向全局集合中真实对象。
2. 找到后只改一处 `isLike`，没有给每个页面复制状态。
3. 三次 `reFresh` 重新构建 UI，保证任一页面点击爱心后三页立即一致。
4. 这一步只改内存；实际入库发生在退出时。

```

void QQMusic::onQQMusicQuit()
{
    musicList.writeToDB();
    sqlite.close();
    close();
}

void QQMusic::on_quit_clicked()
{
    hide();
}

```

1. 托盘“退出”会写 DB、关闭连接、关闭窗口，属于真正退出路径。
2. 标题栏视觉关闭按钮只 `hide()`，程序会继续在托盘运行，符合音乐播放器常见行为。
3. 风险：Alt+F4 或其他 `close()` 路径可能绕过 `onQQMusicQuit()`，最后状态不一定持久化。更可靠的方案是重写 `closeEvent()`。

```

void QQMusic::onBtFormClicked(int pageId)
{
    QList<btForm*> btList = this->findChildren<btForm*>();
    for(auto& btItem : btList)
        if(btItem->getPageId() != pageId) btItem->clearBg();
    ui->stackedWidget->setCurrentIndex(pageId);
    isDrag = false;
}

```

1. 找到所有导航控件。
2. 被点击之外的按钮清除“选中绿色背景”。点击按钮自己已在 `btForm::mousePressEvent` 内设绿。
3. 根据 ID 切换堆叠页。
4. `isDrag = false` 避免点击子控件后被父窗口拖动逻辑误判。

```

void QQMusic::mousePressEvent(QMouseEvent *event)
{
    if(Qt::LeftButton == event->button()) {
        isDrag = true;
        dragPosition = event->globalPos() - geometry().topLeft();
        return;
    }
}

```

```

    QWidget::mousePressEvent(event);
}

void QQMusic::mouseMoveEvent(QMouseEvent *event)
{
    if(Qt::LeftButton == event->button() && isDrag) {
        move(event->globalPos() - dragPosition);
        return;
    }
    QWidget::mouseMoveEvent(event);
}

```

1. 无边框窗口失去系统拖动能力，因此手动实现。
2. `event->button()` 是本次按下的按钮；`event->buttons()` 是移动时仍处于按下状态的按钮集合。
3. `globalPos()` 是屏幕坐标；`geometry().topLeft()` 是窗口当前屏幕左上角。
4. 二者相减得到鼠标相对窗口的固定偏移，拖动时不会使窗口左上角突然吸附到鼠标。
5. 只处理左键事件，其余交还父类保证默认行为。

```

void QQMusic::on_volume_clicked()
{
    QPoint point = ui->volume->mapToGlobal(QPoint(0,0));
    QPoint volumeLeftTop = point - QPoint(volumeTool->width()/2, volumeTool->
    >height());
    volumeLeftTop.setX(volumeLeftTop.x() + 15);
    volumeLeftTop.setY(volumeLeftTop.y() + 30);
    volumeTool->move(volumeLeftTop);
    volumeTool->show();
}

```

1. `mapToGlobal` 将音量按钮局部 `(0,0)` 转成屏幕坐标。
2. 减去浮层半宽和整高，使音量浮层出现在按钮上方且大致居中。
3. 两个 `setX/setY` 是视觉微调常量。
4. `move` 后 `show`，每次点击都重新定位，适应主窗口移动。

```

void QQMusic::on_addLocal_clicked()
{
    QFileDialog* fileDialog = new QFileDialog(this);
    fileDialog->setWindowTitle("添加本地音乐");
    fileDialog->setAcceptMode(QFileDialog::AcceptOpen);
    fileDialog->setFileMode(QFileDialog::ExistingFiles);
    QStringList mimeTypeFilters;
    mimeTypeFilters << "application/octet-stream";
    fileDialog->setMimeTypeFilters(mimeTypeFilters);
}

```

1. 对话框有 `parent`，会随主窗口释放；但本函数结束后也可用栈对象简化。
2. `AcceptOpen` 表示打开而非保存。
3. `ExistingFiles` 允许选择多个已存在文件。
4. `application/octet-stream` 是泛二进制类型，不是“音频文件”；它的过滤效果并不精确，真正类型校验在 `MusicList`。

```
QDir dir(QDir::currentPath());
dir.cdUp();
QString projectPath = dir.path();
projectPath += "/QQMusic/musics/";
fileDialog->setDirectory(projectPath);
```

1. 代码假设当前工作目录是构建目录，并向上一级后能得到项目根目录。
2. 这在 IDE 中可能成立，在安装后或从其他目录启动时并不可靠。
3. 应改用 `QStandardPaths::MusicLocation` 或保存上次目录。

```
if(QDialog::Accepted == fileDialog->exec()) {
    QList<QUrl> fileUrls = fileDialog->selectedUrls();
    musicList.addMusicByUrl(fileUrls);
    ui->stackedWidget->setCurrentIndex(4);
    ui->localPage->refresh(musicList);
    ui->localPage->addMusicToPlayList(musicList, playList);
}
}
```

1. `exec()` 进入该模态对话框自己的事件循环，直到用户确认/取消。
2. 只有确认时获取多选文件 URL。
3. `addMusicByUrl` 完成去重、MIME 检查、元数据解析、内存加入。
4. 导入成功后切回本地页并刷新显示。
5. 最后一行把本地页所有歌追加进 `playList`，但没有先 `clear()`；而真正播放时又会重建列表。这一行容易造成导入多次后的重复媒体，建议删除或先清空并统一由播放入口管理。

8. QQMusic：播放、进度、封面和歌词的完整状态流

```
void QQMusic::onLrcWordClicked()
{
    lrcPage->show();
    lrcAnimal->start();
}
```

1. 先 `show()`，否则隐藏对象的几何动画不可见。
2. 再启动初始化时配置的上滑动画。

```
void QQMusic::onPlayMusic()
{
    if(player->state() == QMediaPlayer::StoppedState) player->play();
    else if(player->state() == QMediaPlayer::PlayingState) player->pause();
    else if(player->state() == QMediaPlayer::PausedState) player->play();
    else qDebug() << player->errorString();
}
```

1. 停止态调用 `play()`；若播放列表没有有效媒体，播放器会自行报错。
2. 播放态调用 `pause()`。
3. 暂停态再次 `play()`，从当前位置继续。
4. 这里不直接设置图标；`stateChanged` 信号统一驱动图标，避免漏掉自动停止等路径。
5. 末尾分支处理未预期状态并输出底层错误描述。

```
void QQMusic::onPlayUpClicked() { playlist->previous(); }
void QQMusic::onPlayDownClicked() { playlist->next(); }
```

1. 两个槽不直接操作 `player`，而是让播放列表改变当前项。
2. 列表当前索引变更会自动反映到播放器，并触发 `currentIndexChanged` 等后续状态链。
3. 随机模式下“上一首”的行为由 Qt 播放列表规则决定，不是项目自行记录历史。

```
if(playlist->playbackMode() == QMediaPlaylist::Loop) {
    playlist->setPlaybackMode(QMediaPlaylist::Random);
    ui->playMode->setToolTip("随机播放");
} else if(... Random) {
    playlist->setPlaybackMode(QMediaPlaylist::CurrentItemInLoop);
    ui->playMode->setToolTip("单曲循环");
} else if(... CurrentItemInLoop) {
    playlist->setPlaybackMode(QMediaPlaylist::Loop);
    ui->playMode->setToolTip("列表循环");
}
```

1. 模式循环为 `Loop -> Random -> CurrentItemInLoop -> Loop`。
2. `setPlaybackMode` 是状态源；按钮 tooltip 只是辅助文字。
3. 图标不在这里改，而在 `onPlaybackModeChanged` 中响应模式改变。这种“修改状态”和“渲染状态”分离是正确的。
4. 未支持模式仅打印调试信息。

```
void QQMusic::onPlayStateChanged()
{
    if(player->state() == QMediaPlayer::PlayingState)
        ui->play->setIcon(QIcon(":/images/play_on.png"));
    else
        ui->play->setIcon(QIcon(":/images/play3.png"));
}
```

1. 信号未携带参数时直接读取播放器当前状态。
2. 播放时使用一种图标，暂停/停止共用另一种。
3. 因而 UI 始终反映真实播放器状态，而不依赖上一次点击来源。

```
void QQMusic::onPlaybackModeChanged(QMediaPlaylist::PlaybackMode mode)
{
    if(mode == QMediaPlaylist::Loop) ui->playMode->setIcon(...list_play.png);
    else if(mode == QMediaPlaylist::Random) ui->playMode->
>setIcon(...shuffle_2.png);
    else if(mode == QMediaPlaylist::CurrentItemInLoop)
        ui->playMode->setIcon(...single_play.png);
}
```

1. 槽参数就是信号提供的最新模式，避免再次查询。
2. 三种业务模式分别映射三个资源图标。
3. 该函数只处理视觉，不改变模式本身。

```

void QQMusic::onPlayAll(PageType pageType)
{
    CommonPage* page = ui->localPage;
    switch(pageType) {
        case LIKE_PAGE: page = ui->likePage; break;
        case LOCAL_PAGE: page = ui->localPage; break;
        case HISTORY_PAGE: page = ui->recentPage; break;
        default: qDebug() << "暂不支持,待扩展";
    }
    playAllOfCommonPage(page, 0);
}

```

1. 先给 `page` 一个可用默认值，避免未初始化指针。
2. `switch` 将抽象 `PageType` 映射到实际页面对象。
3. 点击“全部播放”统一从索引 0 开始。
4. `default` 只打日志仍会播放默认本地页，这是不严谨之处；扩展未知页面时应 `return`。

```

void QQMusic::playAllOfCommonPage(CommonPage *page, int index)
{
    currentPage = page;
    updateBtFormAnimal();
    playList->clear();
    page->addMusicToPlayList(musicList, playList);
    playList->setCurrentIndex(index);
    player->play();
}

```

1. 这是“全部播放”和“双击播放”共同的唯一入口。
2. `currentPage = page` 记录播放来源，后续索引才能映射回正确的歌曲 ID。
3. 更新侧栏动画，视觉标识当前播放来源。
4. `clear()` 丢弃旧播放上下文，保证只播放当前页面范围。
5. `addMusicToPlayList` 依页面类型向队列添加 URL。
6. `setCurrentIndex(index)` 选择起始曲目；若页面无歌曲或索引非法，值会无效。
7. `play()` 启动播放，随后由播放器信号更新所有 UI。
8. 高风险：`clear()` 可能发射 `currentIndexChanged(-1)`，而项目当前索引处理没有防御负数。

```

void QQMusic::onPlayMusicByIndex(CommonPage *page, int index)
{
    playAllOfCommonPage(page, index);
}

```

双击行索引和全部播放共用同一重建队列逻辑，因此不会出现“UI 中双击的是喜欢页歌曲，播放器却仍在本地队列”的错位。

```
void QQMusic::onPlayCurrentIndexChanged(int index)
{
    currentIndex = index;
    const QString musicid = currentPage->getMusicIdByindex(index);
    auto it = musicList.findMusicById(musicid);
    if(it != musicList.end()) it->setIsHistory(true);
    ui->recentPage->refresh(musicList);
}
```

1. 保存新的播放队列索引。
2. 页面内部保存了“筛选后歌曲 ID 的顺序”，所以可用该索引反查 `musicId`。
3. 再由全局 `musicList` 找到真实 `Music`，标记历史。
4. 最近播放页立刻刷新。
5. 缺陷：当索引为 `-1` 时，`getMusicIdByindex(-1)` 当前实现会发生非法访问；必须先写 `if (index < 0) return;`。

```
void QQMusic::setMusicSilence(bool isMuted) { player->setMuted(isMuted); }
void QQMusic::setMusicVolume(int volume) { player->setVolume(volume); }
```

音量控件只发意图，主窗口把意图落到播放器。这避免 `VolumeTool` 直接依赖播放器。

```
void QQMusic::onDurationChanged(qint64 duration)
{
    totalTime = duration;
    ui->totalTime->setText(QString("%1:%2")
        .arg(duration / 1000 / 60, 2, 10, QChar('0'))
        .arg(duration / 1000 % 60, 2, 10, QChar('0')));
}
```

1. 接收毫秒单位总时长并保存。
2. `/1000/60` 是分钟；`/1000%60` 是秒。
3. 两次 `arg` 链式替换 `%1`、`%2`。
4. 宽度 `2`、进制 `10`、补字符 `'0'` 使 `3:7` 显示为 `03:07`。

```
void QQMusic::onPositionChanged(qint64 position)
{
    ui->currentTime->setText(...);
    ui->progressBar->setStep(position / (float)totalTime);
    if(currentIndex >= 0) lrcPage->showLrcWord(position);
}
```

1. 首句同样把当前位置毫秒转换为 `mm:ss`。
2. 强转 `float` 避免整数除法，得到 `0~1` 的比例。
3. `MusicSlider::setStep` 用比例渲染进度。
4. 仅当前索引有效时更新歌词。
5. 若 `totalTime == 0`，会产生无效浮点结果；应先判断 `totalTime > 0`。


```
void QQMusic::onMusicSliderChanged(float ratio)
{
    qint64 durationTime = (qint64)(totalTime * ratio);
    ui->currentTime->setText(...durationTime...);
    player->setPosition(durationTime);
}
```

1. 滑条发出的比例乘总时长，恢复目标毫秒。
2. 先乐观刷新当前时间，随后请求播放器 seek。
3. `setPosition` 成功后播放器还会发 `positionChanged`，UI 会再次被真实位置校正。

```
void QQMusic::onMediaAvailableChanged(bool available)
{
    (void)available;
    QString musicId = currentPage->getMusicIdByindex(currentIndex);
    auto it = musicList.findMusicById(musicId);
    QString musicName("未知歌曲"), musicSigner("未知歌手");
    if(it != musicList.end()) { ...读取歌名和歌手...; }
```

1. `(void)available` 压制未使用参数警告，但也意味着无论元数据是否可用都继续执行；应在 `!available` 时返回。
2. 先由“来源页 + 播放索引”定位歌曲，再从全量集合取业务元数据。
3. 默认值保证未找到歌曲时 UI 仍可显示。

```
QVariant coverImage = player->metaData("ThumbnailImage");
if(coverImage.isValid()) {
    QImage musicImage = coverImage.value<QImage>();
    ui->musicCover->setPixmap(QPixmap::fromImage(musicImage));
    currentPage->setMusicImage(QPixmap::fromImage(musicImage));
} else {
    QString path = ":/images/rec/001.png";
    ui->musicCover->setPixmap(path);
    currentPage->setMusicImage(path);
}
```

1. `metaData` 返回通用 `QVariant`，需先检查 `isValid()`。
2. 有封面时从 `QVariant` 取出 `QImage` 并转为 `QPixmap`。
3. 主播放区和当前 `CommonPage` 头图同时更新。
4. 无封面时使用资源图兜底。
5. `setScaledContents(true)` 随后让封面填满标签，但可能造成图像拉伸；生产版宜使用等比例裁剪。

```
lrcPage->setMusicNameAndSinger(musicName, musicSigner);
if(it != musicList.end()) {
    QString lrcPath = it->getMusicLrcPath();
    lrcPage->parseLrc(lrcPath);
}
}
```

1. 歌词页同步标题和歌手。
2. 歌词路径由歌曲 URL 推导。

3. 当前忽略 `parseLrc` 返回值；当歌词文件不存在时，`LrcPage` 内部没有先清空旧歌词，会残留上一首歌词，违背“无歌词显示空内容”的目标。

架构沉淀

播放器信号是全项目的“时间与状态真相”。`QQMusic` 的主设计是对的：页面请求播放，主窗口重建队列，播放器信号反推 UI。必须补齐负索引、零时长和无歌词三类边界，才能让这条链在异常状态下仍安全。

9. Music：单首歌曲对象、元数据与入库

源码：[music.h](#)、[music.cpp](#)

```
Music::Music() : isLike(false), isHistory(false) {}
```

默认构造用于 DB 反序列化。两种用户状态必须明确为 `false`；其他 `QString/QUrl` 默认空。
`musicDuration` 在默认构造中未初始化，DB 读取前不可使用。

```
Music::Music(const QUrl& url)
    : isLike(false), isHistory(false), musicUrl(url)
{
    musicId = QUuid::createUuid().toString();
    parseMediaMetaData();
}
```

1. 新导入的歌默认未收藏、未进历史。
2. `musicUrl(url)` 保存媒体定位地址。
3. `QUuid::createUuid()` 生成近乎全局唯一 ID，`toString()` 转成 DB 可存的字符串。
4. 构造即解析元数据，使对象一旦加入 `MusicList` 就具备可显示字段。
5. 风险是构造函数做了可能耗时的 I/O 和事件循环等待，批量导入时 UI 易卡顿。

```
void Music::setMusicId(const QString& v) { this->musicId = v; }
void Music::setMusicName(const QString& v) { this->musicName = v; }
void Music::setMusicSinger(const QString& v) { this->musicSinger = v; }
void Music::setMusicAlbumn(const QString& v) { this->musicAlbumn = v; }
void Music::setMusicDuration(qint64 v) { this->musicDuration = v; }
void Music::setIsLike(bool v) { this->isLike = v; }
void Music::setIsHistory(bool v) { this->isHistory = v; }
void Music::setMusicUrl(const QUrl& v) { this->musicUrl = v; }
```

每个 setter 都把形参写入同名成员，`this->` 用来消除名称遮蔽。它们主要被 `readFromDB()` 使用，将一行 SQL 记录还原为对象。`Albumn` 是拼写错误，但头文件和实现一致，不影响编译；新项目应统一为 `Album`。

```

QString Music::getMusicId() const { return musicId; }
QString Music::getMusicName() const { return musicName; }
QString Music::getMusicSinger() const { return musicSinger; }
QString Music::getMusicAlbumn() const { return musicAlbumn; }
qint64 Music::getMusicDuration() const { return musicDuration; }
bool Music::getIsLike() const { return isLike; }
bool Music::getIsHistory() const { return isHistory; }
QUrl Music::getMusicUrl() const { return musicUrl; }

```

1. 每个 getter 都按值返回成员；`QString`、`QUrl` 是隐式共享类型，小对象复制成本通常可接受。
2. 末尾 `const` 承诺不会修改对象，因此可对常量对象调用。
3. 这些函数是页面和列表的只读访问入口，避免外部直接改成员。

```

QString Music::getMusicLrcPath() const
{
    QString lrcPath = musicUrl.toLocalFile();
    lrcPath.replace(".mp3", ".lrc");
    lrcPath.replace(".flac", ".lrc");
    lrcPath.replace(".mpga", ".lrc");
    return lrcPath;
}

```

1. `toLocalFile()` 把本地 URL 转回磁盘路径。
2. 三次 `replace` 依据“歌词与音频同名且只差扩展名”的约定推导 LRC。
3. 只支持小写扩展名，且会替换路径中任意匹配文本；更稳妥是 `QFileInfo` 重建扩展名。

```

void Music::insertToDB()
{
    QSqlQuery query;
    query.prepare("SELECT EXISTS(SELECT 1 FROM MusicInfo WHERE musicId = ?)");
    query.addBindValue(musicId);
    if(!query.exec()) { qDebug() << ...; return; }
}

```

1. 每首歌退出时调用一次该函数。
2. `prepare` 使用 `?` 占位，避免直接拼接 SQL。
3. `addBindValue` 按占位顺序绑定 UUID。
4. `SELECT EXISTS` 只返回 0/1，比取整行记录更符合“是否存在”的目的。

```

    if(query.next()) {
        bool isExists = query.value(0).toBool();
        if(isExists) {
            query.prepare("UPDATE MusicInfo SET isLike = ?, isHistory = ? WHERE musicId = ?");
            query.addBindValue(isLike ? 1 : 0);
            query.addBindValue(isHistory ? 1 : 0);
            query.addBindValue(musicId);
            if(!query.exec()) { ... return; }
        } else {
            query.prepare("INSERT INTO MusicInfo(...) VALUES(?, ?, ?, ?, ?, ?, ?, ?)");
            // 依次绑定 id、名称、歌手、专辑、路径、时长、喜欢、历史
            if(!query.exec()) { ... return; }
        }
    }
}

```

```

    }
}
}

```

1. `next()` 将查询游标移到唯一结果行。
2. 已存在时仅更新可变状态 `isLike/isHistory`，静态元数据保持首次入库值。
3. 新记录时依 SQL 列顺序绑定八个字段，URL 使用 `toLocalFile()` 保存普通路径。
4. 三元表达式将 `bool` 显式转为 SQLite 整数 0/1。
5. 该“先查再写”在单进程 Demo 可行；更简洁的 SQLite 写法是 `INSERT ... ON CONFLICT(musicId) DO UPDATE`。

```

QMediaPlayer player;
player.setMedia(musicUrl);
while(!player.isMetaDataAvailable()) QApplication::processEvents();

```

这是 `parseMediaMetaData()` 的核心。

1. 临时播放器只用于让 Qt 多媒体后端解析文件。
2. `setMedia` 发起异步媒体加载。
3. `isMetaDataAvailable()` 初始通常为假。
4. `processEvents()` 在 `while` 内手动处理事件，避免主事件循环完全冻结。
5. 但没有超时、错误状态或取消条件；损坏文件/后端不支持时可能无限循环。这是当前最严重的导入稳定性问题。

```

musicName = player.metaData("Title").toString();
musicSinger = player.metaData("Author").toString();
musicAlbumn = player.metaData("AlbumTitle").toString();
musicDuration = player.metaData("Duration").toLongLong();

```

四行从容器格式元数据读取标题、作者、专辑和毫秒时长。文件不带标签时，会得到空字符串或零。

```

QString fileName = musicUrl.fileName();
int index = fileName.indexOf('-');
if(musicName.isEmpty()) {
    if(index != -1) musicName = fileName.mid(0, index).trimmed();
    else musicName = fileName.mid(0, fileName.indexOf('.')).trimmed();
}
if(musicSinger.isEmpty()) {
    if(index != -1)
        musicSinger = fileName.mid(index + 2, fileName.indexOf('.') - index - 2).trimmed();
    else musicSinger = "未知歌手";
}
if(musicAlbumn.isEmpty()) musicAlbumn = "未知专辑";

```

1. 元数据缺失时降级解析文件名，例如 `歌名 - 歌手.mp3`。
2. `indexOf('-')` 找第一个连字符；项目通过 `+2/-2` 假设两侧都有空格。
3. `trimmed()` 去除两端空白，保证表格列对齐。
4. 文件名没有连字符时，歌名取扩展名前内容，歌手给默认值。
5. 含多个 `-`、没有扩展名、使用全角横线、带额外后缀的文件会解析不准确；这只是展示级兜底，不是可靠媒体标签方案。

架构沉淀

`Music` 是值对象，封装一首歌的静态信息和用户状态，配合 UUID 让页面、数据库和播放来源能稳定关联。主要改进目标是将异步元数据解析移出构造函数，并对加载超时、错误和路径变化做处理。

10. `MusicList`：全量集合、去重与数据库恢复

源码：[musiclist.h](#)、[musiclist.cpp](#)

```
typedef QVector<Music>::iterator iterator;
QVector<Music> musicList;
QSet<QString> musicPaths;
```

1. `typedef` 简化迭代器写法，但在现代 C++ 中可用 `using Iterator = ...`。
2. `QVector<Music>` 按值保存全部歌曲，支持顺序遍历和索引。
3. `QSet<QString>` 保存本地路径，实现 $O(1)$ 平均复杂度去重。

```
void MusicList::addMusicByUrl(const QList<QUrl>& musicUrls)
{
    for(auto& musicurl : musicUrls) {
        QString musicPath = musicurl.toLocalFile();
        if(musicPaths.contains(musicPath)) continue;
        musicPaths.insert(musicPath);
        QMimeDatabase mimeDB;
        QMimeType mimeType = mimeDB.mimeTypeForFile(musicPath);
        QString mime = mimeType.name();
        if(mime == "audio/mpeg" || mime == "audio/flac" || mime == "audio/wav")
        {
            Music music(musicurl);
            musicList.push_back(music);
        }
    }
}
```

1. 形参以常量引用传入，避免复制 URL 列表。
2. range-for 的 `auto&` 避免复制每个 `QUrl`。
3. URL 转本地路径，作为去重键。
4. 已存在则 `continue` 跳过本轮，避免重复导入和重复播放器媒体。
5. 当前代码在 MIME 校验前就 `insert` 路径；用户若先选到非音频文件，该路径会被永久标记，虽然通常无害，但顺序不严谨。
6. `QMimeDatabase` 按文件类型识别格式，过滤 MP3、FLAC、WAV。
7. 合法时构造 `Music`，该过程会同步解析元数据，然后按值压入向量。

```
void MusicList::readFromDB()
{
    QSqlQuery query;
    query.prepare("SELECT musicId, musicName, musicSinger, albumName, musicUrl,
duration, isLike, isHistory FROM MusicInfo");
    if(!query.exec()) { ... return; }
    while(query.next()) {
        Music music;
```

```

        music.setMusicId(query.value(0).toString());
        ...
        music.setMusicUrl(QUrl::fromLocalFile(query.value(4).toString()));
        ...
        musicList.push_back(music);
        musicPaths.insert(music.getMusicUrl().toLocalFile());
    }
}

```

1. 查询列顺序必须与后面的 `value(0)~value(7)` 对齐。
2. 每次 `next()` 移到一行，创建默认 `Music`，逐字段 setter 恢复。
3. URL 必须通过 `fromLocalFile` 重建；直接 `QUrl(path)` 会把 Windows 路径当普通 URL，影响播放和歌词路径。
4. 每首恢复的歌加入 `musicList`，路径也加入 `musicPaths`，防止本次运行再导入同文件。
5. 当前没有 `QFileInfo::exists` 校验，文件被移动/删除后仍会显示在本地列表中。

```

void MusicList::writeToDB()
{
    for(auto& music : musicList) music.insertToDB();
}

```

遍历内存全量集合，将每个对象交给 `Music::insertToDB()` 做插入或状态更新。`auto&` 很关键，避免复制歌曲；但写入没有事务，很多歌曲时中途失败会留下部分更新。

```

iterator MusicList::findMusicById(const QString& id)
{
    for(iterator it = begin(); it != end(); ++it)
        if(it->getMusicId() == id) return it;
    return end();
}
iterator MusicList::begin() { return musicList.begin(); }
iterator MusicList::end() { return musicList.end(); }

```

1. 线性查找 UUID，复杂度 $O(n)$ 。
2. 找到返回可修改迭代器，失败返回尾迭代器，这是 STL/Qt 容器标准约定。
3. `begin/end` 将底层 `QVector` 遍历能力暴露给页面，但没有暴露整个容器。
4. 小型 Demo 足够；大量歌曲应使用 `QHash<QString, int>` 或模型层索引。

架构沉淀

`MusicList` 做到了“集合与去重”职责集中。值得保留；应改善 MIME 校验插入顺序、失效路径处理、事务写入和 ID 索引性能。

11. CommonPage 与 ListItemBox：三种歌曲页的一份实现

源码：[commonpage.cpp](#)、[listitembox.cpp](#)

```

enum PageType { LIKE_PAGE, LOCAL_PAGE, HISTORY_PAGE };

```

这是页面筛选策略的枚举。当前是普通 enum，值会隐式转换为整数；更严格的代码应使用 `enum class PageType`。

```
CommonPage::CommonPage(QWidget *parent) : QWidget(parent), ui(new
Ui::CommonPage)
{
    ui->setupUi(this);
    ui->pageMusicList->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);
    connect(ui->playAllBtn, &QPushButton::clicked, this, [=]() { emit
playAll(pageType); });
    connect(ui->pageMusicList, &QListWidget::doubleClicked, this,
        [=](const QModelIndex &index) { emit playMusicByindex(this, index.row());
});
}
```

1. 先构造 QWidget，再创建和安装 Designer UI。
2. 关闭水平滚动条，列表行宽由设计控制。
3. “全部播放”点击时，lambda 捕获 `this`，把自己的 `pageType` 发给主窗口。
4. 列表双击时，`QModelIndex::row()` 是当前可见筛选列表的行号。
5. 信号附带 `this`，因为主窗口必须知道这个索引属于喜欢、本地还是最近页。

```
void CommonPage::setPageType(PageType v) { pageType = v; }
void CommonPage::setCommonPageUI(const QString& text, const QString& imagePath)
{
    ui->pageTitle->setText(text);
    ui->musicImageLabel->setPixmap(QPixmap(imagePath));
    ui->musicImageLabel->setScaledContents(true);
}
```

1. setter 写入筛选策略。
2. UI 注入设置标题与头图。
3. `setScaledContents(true)` 强制图片填满标签，简单但可能压缩变形。

```
void CommonPage::addMusicToMusicPage(MusicList& list)
{
    musicListOfPage.clear();
    for(auto& music : list) {
        switch(pageType) {
            case LIKE_PAGE: if(music.getIsLike())
musicListOfPage.push_back(music.getMusicId()); break;
            case LOCAL_PAGE: musicListOfPage.push_back(music.getMusicId()); break;
            case HISTORY_PAGE: if(music.getIsHistory())
musicListOfPage.push_back(music.getMusicId()); break;
            default: qDebug() << "暂不支持";
        }
    }
}
```

1. 每次筛选前清空旧 ID，防止刷新时重复。
2. 遍历的是全量集合。
3. 喜欢页仅压入已收藏歌曲 ID。
4. 本地页无条件压入全部 ID。

5. 最近页仅压入历史标记为真的 ID。
6. 页面存 ID 而非 `Music` 副本，确保状态只有一个真实来源。

```
void CommonPage::reFresh(MusicList &list)
{
    ui->pageMusicList->clear();
    addMusicToMusicPage(list);
    for(auto& musicId : musicListOfPage) {
        auto it = list.findMusicById(musicId);
        if(it == list.end()) continue;
        ListItemBox* box = new ListItemBox(this);
        box->setMusicName(it->getMusicName());
        box->setMusicSinger(it->getMusicSinger());
        box->setMusicAlbum(it->getMusicAlbumn());
        box->setLikeMusic(it->getIsLike());
        QListWidgetItem* item = new QListWidgetItem(ui->pageMusicList);
        item->setSizeHint(QSize(box->width(), box->height()));
        ui->pageMusicList->setItemWidget(item, box);
        connect(box, &ListItemBox::setLike, this,
            [=](bool isLike){ emit updateLikeMusic(isLike, it->getMusicId()); });
    }
    repaint();
}
```

1. `clear()` 删除列表项及其关联展示，避免重复显示。
2. 重算当前页 ID 列表。
3. 每个 ID 再回到全局 `MusicList` 找真实对象；找不到就跳过，保证容错。
4. 创建一行 `ListItemBox`，写入歌曲名、歌手、专辑和收藏图标。
5. 创建 `QListWidgetItem` 作为容器项，设置尺寸提示后使用 `setItemWidget` 嵌入自定义行控件。
6. 行内爱心信号被 lambda 截获并转发，连同闭包捕获的歌曲 ID 上交主窗口。
7. `repaint()` 是同步重绘，数据量大时会阻塞；一般更推荐 `update()`，由事件循环合并刷新。

```
void CommonPage::addMusicToPlayList(MusicList &list, QMediaPlaylist *playlist)
{
    for(auto& music : list) {
        switch(pageType) {
            case LIKE_PAGE: if(music.getIsLike()) playlist-
>addMedia(music.getMusicUrl()); break;
            case LOCAL_PAGE: playlist->addMedia(music.getMusicUrl()); break;
            case HISTORY_PAGE: if(music.getIsHistory()) playlist-
>addMedia(music.getMusicUrl()); break;
            default: qDebug() << "暂不支持";
        }
    }
}
```

这段筛选条件与 `addMusicToMusicPage` 必须完全一致：UI 的第 N 行才会对应播放列表的第 N 项。它把 URL 加入队列，不复制 `Music` 对象。


```
const QString CommonPage::getMusicIdByindex(int index)
{
    if(index >= musicListOfPage.size()) return "";
    return musicListOfPage[index];
}
```

1. 超出上界返回空字符串。
2. **没有检查** `index < 0`。-1 会绕过判断并非法访问，这是播放列表清空时的真实崩溃风险。
3. 返回值的 `const` 对按值返回类型没有实际意义，可去掉。

```
ListItemBox::ListItemBox(...) : QWidget(parent), ui(new Ui::ListItemBox),
isLike(false)
{
    ui->setupUi(this);
    connect(ui->likeBtn, &QPushButton::clicked, this,
    &ListItemBox::onLikeBtnClicked);
}
```

初始化爱心状态为未收藏，安装 UI，并把按钮点击连接到自身槽。

```
void ListItemBox::setMusicName(const QString& v) { ui->musicNameLabel-
>setText(v); }
void ListItemBox::setMusicSinger(const QString& v) { ui->musicSingerLabel-
>setText(v); }
void ListItemBox::setMusicAlbum(const QString& v) { ui->musicAlbumLabel-
>setText(v); }
```

三个 setter 分别把显示字段写入 Designer 标签，没有保存歌曲数据；真正数据仍在 `MusicList`。

```
void ListItemBox::setLikeMusic(bool value)
{
    isLike = value;
    if(isLike) ui->likeBtn->setIcon(QIcon(":/images/like_2.png"));
    else ui->likeBtn->setIcon(QIcon(":/images/like_3.png"));
}
void ListItemBox::onLikeBtnClicked()
{
    isLike = !isLike;
    setLikeMusic(isLike);
    emit setLike(isLike);
}
```

1. `setLikeMusic` 同步内部布尔值与图标。
2. 点击时先取反，再复用 setter 更新视觉。
3. 最后只发状态，不直接写数据库；上层将它与具体歌曲 ID 重新关联。

```
void ListItemBox::enterEvent(QEvent *) { setStyleSheet("background-
color:#EFEFEF"); }
void ListItemBox::leaveEvent(QEvent *) { setStyleSheet(""); }
```

重写鼠标进入/离开事件实现 hover 背景。参数未使用，因此省略变量名或 `(void)event` 都是抑制警告的方法。

架构沉淀

`CommonPage` 是“筛选投影”，`ListWidgetItem` 是“单行视图”。这种分离值得学习。当前 `QListWidget + setItemWidget` 适合小数据量；歌曲很多时应升级到 `QAbstractListModel + QListView`。

12. LrcPage：歌词文件到同步显示

源码：[lrcpage.cpp](#)

```
LrcPage::LrcPage(...) : QWidget(parent), ui(new Ui::LrcPage)
{
    ui->setupUi(this);
    setWindowFlag(Qt::FramelessWindowHint);
    animal = new QPropertyAnimation(this, "geometry", this);
    animal->setDuration(500);
    animal->setStartValue(QRect(10, 10, width(), height()));
    animal->setEndValue(QRect(10, 10 + height(), width(), height()));
    connect(ui->hideBtn, &QPushButton::clicked, this, [=]{ animal->start(); });
    connect(animal, &QPropertyAnimation::finished, this, [=]{ hide(); });
    showLrcWord(-1);
}
```

1. 安装 UI 后设无边框。
2. `animal` 是收起动画，与主窗口的展开动画相反：从正常位置移动到下方。
3. 隐藏按钮点击只启动动画；动画完成后才 `hide()`，否则看不到收起过程。
4. `showLrcWord(-1)` 初始化“暂无歌词”的界面状态。

```
void LrcPage::setMusicNameAndSinger(QString &name, QString &singer)
{
    ui->musicName->setText(name);
    ui->musicSinger->setText(singer);
}
```

两个参数仅读取，却写成非常量引用；应改成 `const QString&`，表达正确意图并允许传入临时值。

```
bool LrcPage::parseLrc(const QString &lrcPath)
{
    QFile lrcFile(lrcPath);
    if(!lrcFile.open(QFile::ReadOnly)) return false;
    lrcLines.clear();
    while(!lrcFile.atEnd()) { ... }
    return true;
}
```

1. 根据本地路径创建文件对象。
2. 打不开就失败返回。
3. **清空发生在打开成功之后**，所以新歌无歌词时，旧歌歌词会残留；应把 `lrcLines.clear()` 放到打开前。
4. 成功时逐行读到 EOF。

```
QString word = lrcFile.readLine(1024);
int start = 0, end = word.indexOf(']', start);
QString lrcTime = word.mid(start, end - start + 1);
QString lrcText = word.mid(end + 1, word.size() - end - 2);
```

1. 每次最多读取 1024 字节，超长歌词行会被拆开。
2. 查找第一个]，假设格式是 [mm:ss.xx]文本。
3. 时间子串包含方括号，文本从] 后开始，末尾 -2 试图去掉换行。
4. 未验证] 是否存在，也不支持一行多个时间标签。

```
start = 1; end = word.indexOf(':', start);
lineTime += lrcTime.mid(start, end-start).toLong() * 60 * 1000;
start = end + 1; end = word.indexOf('.', start);
lineTime += lrcTime.mid(start, end-start).toLong() * 1000;
start = end + 1; end = word.indexOf('.', start);
lineTime += lrcTime.mid(start, end-start).toLong();
```

1. 第一段取分钟并转毫秒。
2. 第二段取秒并转毫秒。
3. 第三段本应解析小数部分，但又从相同 start 查同一个 .，得到相同位置，mid(..., 0) 通常为 0，毫秒会丢失。
4. 因此 [00:17.94] 实际接近按 17 秒处理，不会精确同步。
5. 推荐用正则捕获 ^(?:[(\d+):(\d+(?:\.\d+)?)]) (.+)\$，并将小数秒转换为毫秒。

```
void LrcPage::showLrcWord(qint64 time)
{
    int index = getWordIndex(time);
    if(index == -1) { 清空六行并显示“该歌曲暂无歌词”; }
    else { 依次显示 index-3 到 index+3; }
}
```

1. getWordIndex 根据播放时间返回当前行。
2. 无歌词时，中心行显示提示，周围清空。
3. 有歌词时将前 3、当前、后 3 行写到七个标签。
4. 下标边界由 getLrcWordByIndex 统一保护。

```
int LrcPage::getWordIndex(qint64 pos)
{
    if(lrcLines.isEmpty()) return -1;
    if(pos <= lrcLines[0]._wordTime) return 0;
    for(int i = 1; i < lrcLines.size(); ++i)
        if(pos >= lrcLines[i-1]._wordTime && pos < lrcLines[i]._wordTime) return i-1;
    return lrcLines.size()-1;
}
```

1. 空数组没有当前行。
2. 首行之前显示首行。
3. 遍历相邻时间区间，命中时显示前一行。
4. 播放超过最后时间时持续显示最后一行。
5. 算法是 O(n)，歌词通常几十到几百行足够；高频位置更新可缓存上次索引或二分查找。

```
QString LrcPage::getLrcWordByIndex(int index)
{
    if(index < 0 || index >= lrcLines.size()) return "";
    return lrcLines[index]._wordText;
}
```

这是前后 3 行显示的边界保护：超出歌词范围返回空字符串，因此首行不会访问负下标。

架构沉淀

歌词展示与播放器解耦得不错：页面只接受时间与文本。但解析器假设过强，且毫秒解析错误；这是最适合你练习重构为“独立 LRC 解析器”的模块。

13. 其余自定义控件：侧栏、推荐、进度和音量

13.1 btForm：带播放动态条的侧栏按钮

源码：[btform.cpp](#)

```
ui->lineBox->hide();
line1Animat = new QPropertyAnimation(ui->line1, "geometry", this);
line1Animat->setDuration(1500);
line1Animat->setKeyValueAt(0, QRect(7,25,2,0));
line1Animat->setKeyValueAt(0.5, QRect(7,10,2,15));
line1Animat->setKeyValueAt(1, QRect(7,25,2,0));
line1Animat->setLoopCount(-1);
line1Animat->start();
```

1. `lineBox` 是装四根竖线的容器，初始隐藏。
2. `line1` 的 `geometry` 动画从高度 0 拉到 15 再缩回 0。
3. `setKeyValueAt` 的 0、0.5、1 是动画进度比例。
4. `-1` 表示永久循环。
5. `line2~line4` 重复完全相同逻辑，只是 X 坐标和时长分别不同，形成错峰跳动效果。
6. 动画即使容器隐藏也已经启动；可在 `showAnimat/hideAnimat` 中启动和暂停以节约资源。

```
void btForm::setIconAndText(const QString &icon, const QString &text, int id)
{
    ui->btIcon->setPixmap(QPixmap(icon));
    ui->btText->setText(text);
    pageId = id;
}
int btForm::getPageId() const { return pageId; }
void btForm::clearBg() { ui->btStyle->setStyleSheet("#btStyle:hover{background-color:#D8D8D8;}"); }
void btForm::showAnimat() { ui->lineBox->show(); }
void btForm::hideAnimat() { ui->lineBox->hide(); }
```

1. 三个参数分别写入显示图标、显示文字、导航页 ID。
2. getter 暴露 ID 给主窗口比对。
3. `clearBg` 恢复仅 hover 时变灰的样式。
4. `show/hide` 只控制容器可见性，动画本身持续运行。

```
void btForm::mousePressEvent(QMouseEvent *event)
{
    (void)event;
    ui->btStyle->setStyleSheet("#btStyle { background-color:rgb(30,206,154);
}");
    emit btClick(pageId);
}
```

1. 事件参数不参与逻辑。
2. 按下即设为绿色选中态。
3. 发出页 ID，由 `QQMusic` 统一切页和清理其余按钮。

13.2 RecBox / RecBoxItem：推荐卡片分页与 hover 动画

```
RecBox::RecBox(...) : QWidget(parent), ui(new Ui::RecBox), row(1), col(4) { ui-
>setupUi(this); }
```

默认布局为一行四列。

```
void RecBox::initRecBoxUi(QJsonArray data, int row)
{
    if(row == 2) { this->row = row; col = 8; }
    else ui->recListDown->hide();
    imageUrl = data;
    currentIndex = 0;
    count = imageUrl.size() / col;
    createRecItem();
}
```

1. 两行模式把每页数量改为 8；一行模式隐藏下排。
2. 保存传入 JSON 数据、从第 0 页开始。
3. `count` 用整除计算页数。39 张图按 4 张时只会得到 9 页，最后 3 张不会显示；应使用向上取整。
4. 最后创建当前页卡片。

```
void RecBox::createRecItem()
{
    findChildren<RecBoxItem*>() 后从布局 removewidget 并 delete;
    for(int i=currentIndex*col; i<col+currentIndex*col; ++i) {
        RecBoxItem* item = new RecBoxItem();
        QJsonObject object = imageUrl[i].toObject();
        item->setRecText(object.value("text").toString());
        item->setRecImage(object.value("path").toString());
        根据 row/index 添加到上或下布局;
    }
}
```

1. 先删除上一批卡片，避免布局堆积和内存泄漏。
2. 根据当前页索引计算本页数据区间。
3. 创建卡片、从 JSON 取文字与图片路径、写入 UI。
4. 两行模式的后半卡片加入下方布局，前半加入上方。
5. 这里假设总图片数能被 `col` 整除；结合前面的整除页数避免越界，却丢弃尾项。

```

void RecBox::on_btDown_clicked()
{
    ++currentIndex;
    if(currentIndex >= count) currentIndex = 0;
    createRecItem();
}
void RecBox::on_btUp_clicked()
{
    --currentIndex;
    if(currentIndex < 0) currentIndex = count - 1;
    createRecItem();
}

```

前后翻页都采用环形索引，到尾回首、到首回尾。

```

ui->musicImageBox->installEventFilter(this);
bool RecBoxItem::eventFilter(QObject *watched, QEvent *event)
{
    if(watched == ui->musicImageBox) {
        if(event->type() == QEvent::Enter) 创建 150ms 上移动画;
        else if(event->type() == QEvent::Leave) 创建 150ms 下移动画;
        connect(animat, &QPropertyAnimation::finished, this, [=]{ delete animat;
    });
        return true;
    }
    return QObject::eventFilter(watched, event);
}

```

1. 安装过滤器后，卡片对象能收到子控件的进入/离开事件。
2. 进入时将图片框由 Y=9 移到 Y=0，离开反向移动。
3. 动画每次动态创建，完成时显式删除，避免长期泄漏。
4. 返回 `true` 表示事件已消费；其他对象交还基类。
5. `setRecText` 调标签文字；`setRecImage` 拼接 `border-image:url(...)` 样式写入图片容器。

13.3 MusicSlider：鼠标 X 坐标到 seek 比例

源码：[musicslider.cpp](#)

```

currentPos = 0;
maxwidth = width();
moveSlider();

```

初始进度为 0，缓存当前控件宽度并按其绘制填充线。注意没有处理 `resizeEvent`，窗口缩放后 `maxwidth` 可能过期。

```

void MusicSlider::moveSlider()
{
    ui->outline->setGeometry(ui->outline->x(), ui->outline->y(), currentPos, ui->outline->height());
}
void MusicSlider::checkCurrentPos()
{
    if(currentPos < 0) currentPos = 0;
    if(currentPos > maxWidth) currentPos = maxWidth;
}

```

第一段只改变填充条宽度；第二段把位置夹在 `[0,maxwidth]`，是所有鼠标输入后的边界保护。

```

void MusicSlider::setStep(float ratio)
{
    currentPos = maxWidth * ratio;
    moveSlider();
}

```

播放器位置用比例映射成像素宽度。调用者应保证 `ratio` 在 `[0,1]`，当前函数未自行夹紧。

```

mousemoveEvent: currentPos = event->pos().x(); checkCurrentPos(); moveSlider();
mousePressEvent: 检查本控件矩形与左键，再按相同流程更新；
mouseReleaseEvent: 更新位置后 emit
setMusicSliderPosition(currentPos/(float)maxwidth);

```

1. `pos()` 是相对于该滑条的局部坐标，正适合直接作为填充宽度。
2. 按下/移动即时更新视觉。
3. 松开时才发比例，主窗口据此 seek，避免每个鼠标移动事件都频繁调用播放器。
4. 若 `maxwidth` 为 0 会除零；通常 Designer 布局不会为 0，但应防御。

13.4 VolumeTool：鼠标 Y 坐标到音量

源码：[volumetool.cpp](#)

```

setWindowFlags(Qt::Popup | Qt::FramelessWindowHint |
Qt::NoDropShadowWindowHint);
setAttribute(Qt::WA_TranslucentBackground);

```

`Qt::Popup` 使其成为点击外部会自动关闭的弹出窗；无边框和透明背景允许自绘三角尖角及自定义阴影。

```

isMuted(false), volumeRatio(20)
ui->silenceBtn->setIcon(QIcon(":/images/volumn.png"));
ui->volumeRatio->setText("20%");

```

内部状态和初始 UI 都设为 20%、非静音，必须保持一致。

```
void VolumeTool::paintEvent(QPaintEvent *)
{
    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing, true);
    painter.setPen(Qt::NoPen);
    painter.setBrush(Qt::white);
    QPolygon polygon; polygon.append(a); polygon.append(b); polygon.append(c);
    painter.drawPolygon(polygon);
}
```

1. 重写绘制事件画出浮层底部三角。
2. 抗锯齿、无轮廓、白色填充，使三角和白色面板衔接。
3. 三个点定义顶点；这里使用硬编码坐标，改变 UI 尺寸时需要同步调整。

```
int height = ui->volumeBox->mapFromGlobal(QCursor().pos()).y();
height = height < 25 ? 25 : height;
height = height > 205 ? 205 : height;
ui->outSlider->setGeometry(..., height, ..., 205-height);
ui->sliderBtn->move(..., height-ui->sliderBtn->height()/2);
volumeRatio = (int)(ui->outSlider->height() / (float)180 * 100);
ui->volumeRatio->setText(QString::number(volumeRatio) + "%");
```

1. 从全局鼠标坐标映射到音量区域局部 Y 坐标。
2. 用三元表达式将可移动范围夹在 25 到 205。
3. 填充条从点击高度向下延伸；越靠上，填充越高、音量越大。
4. 滑块按钮定位在填充顶端。
5. 180 是有效轨道高度，因此将像素高度比例换算为 0~100。
6. 这些 25/205/180 都是 UI 硬编码常量，重构时应由控件几何计算。

eventFilter 中：MouseButtonPress 调 calculateVolume；
 MouseButtonRelease 发 setMusicVolume(volumeRatio)；
 MouseMove 调 calculateVolume 并持续发音量；
 ononSilenceBtnClicked 中反转 isMuted、切图标、emit setsilence(isMuted)。

1. 过滤器只处理 volumeBox，其他对象交回基类。
2. 按下立即更新视觉，拖动时连续更新真实音量，松开再补一次确认。
3. 静音不改变 volumeRatio，仅调用播放器的 setMuted；取消静音后保留原音量，符合常见播放器行为。

架构沉淀

这些控件普遍遵守“本地交互和视觉在控件内部，业务副作用发信号给 QQMusic”的原则。它们的主要技术债是硬编码几何、缺少 resize 适配，以及动画/列表尾项的边界处理。

14. 必须掌握的缺陷清单与面试表达

1. **负索引崩溃风险**：播放列表 clear() 可能产生 currentIndexChanged(-1)；
 CommonPage::getMusicIdByindex 没有检查负值。修复：if (index < 0 || index >= size) return {}；并在 onPlayCurrentIndexChanged 中先返回。
2. **歌词残留**：parseLrc 打不开文件时不清旧数组。修复：函数第一句 lrcLines.clear()。

3. **歌词毫秒解析错误**：第二次 `indexOf('.')` 使用相同起点。修复为读取 `.` 后的剩余数字并按位数归一化为毫秒。
4. **元数据解析无限等待**： `while(!isMetaDataAvailable()) processEvents()` 无超时。修复：使用异步信号、错误信号和 `QTimer` 超时。
5. **未初始化状态**：初始化列表补充 `totalTime(0), isDrag(false)`。
6. **数据库写入路径不完整**：只在托盘退出保存。修复：重写 `closeEvent` 或在析构前保证写入。
7. **页面与播放队列潜在不一致**： `addMusicToPlaylist` 的筛选条件必须永远与显示筛选一致；目前一致，但应抽取统一的 `filteredMusicIds()` 防止以后改一处漏一处。
8. **同步重建 UI**：收藏后重建三页，歌曲多时会卡顿。修复方向是 Model/View 与增量更新。

面试时可这样概括项目架构：

我把歌曲数据收敛在 `MusicList`，三个页面只按 `PageType` 保存筛选后的歌曲 ID。用户双击或全部播放时，由主窗口依据来源页重建 `QMediaPlaylist`；播放器的状态、位置和元数据变化再通过 Qt 信号统一驱动播放按钮、进度、历史记录、封面和歌词。这样页面不直接控制播放器，也不会各自维护重复歌曲状态。

15. 推荐复习顺序

1. 在 IDE 中对照本文，从 `main.cpp` -> `qqmusic.cpp` 走一遍启动和播放链。
2. 手动模拟“导入歌曲 -> 收藏 -> 从喜欢页双击 -> 切下一首 -> 退出重启”的状态变化。
3. 先修复本章缺陷清单的前四项，再练习把 LRC 解析与数据库访问拆成独立类。
4. 最后将 `QListWidget` 重构为 `QListView + QAbstractListModel`，这是从 Demo 走向工程化最价值的一步。